Flappy Bird Using Irvine32

Group Members

K241033 Raed Ovais

K240975 Rahim Hasam

K240851 Saad Tahir

Submission Date

23/11/2025

1. Executive Summary

- **Overview:**

The goal of this project was to create a console-based *Flappy Bird* game using MASM and the Irvine32 library. We aimed to replicate the core mechanics of the game while adding features like color, sound effects, and keyboard input handling within a text-mode console.

Key Findings:

This project introduced us to useful, lesser-known MASM functions like gotoxy, which helped position the cursor for precise screen updates. We also incorporated basic sound effects using the beep function and improved our understanding of real-time keyboard input handling.

2. Introduction

Background:

Creating a game in assembly language requires a deep understanding of how the operating system interacts with the assembler and how different system features are linked at the

hardware level. This was especially true for developing the *Flappy Bird* game. Despite being limited to the Irvine32 library, we gained valuable insights into how Irvine32 interfaces with the Windows API and how these interactions enable low-level game functionality.

Project Objectives:

The primary objective was to develop an interactive, graphical *Flappy Bird* game in assembly using the Irvine32 library, focusing on creating a smooth gameplay experience while working within the constraints of assembly language and low-level system resources.

3. Project Description

Inclusions:

- Irvine32 library
- VS Code IDE
- MSVC `ml.exe` assembler

Exclusions:

- Windows API (WinAPI)

4. Methodology

Approach:

We adopted an incremental approach to development, introducing a new feature each week. Each team member was responsible for contributing to a specific feature, ensuring steady progress while maintaining a collaborative workflow. This approach allowed us to

tackle different aspects of the game and refine each feature as the project progressed.

Roles and Responsibilities:

- **Raed:** Responsible for implementing the core game mechanics, including collision detection and the player's movement logic. Raed also worked on optimizing the game loop for smoother gameplay.

- **Rahim:** Focused on integrating visual elements, such as managing screen rendering and adding color. Rahim also implemented the background and pipe movement animations.

- **Saad:** Handled user input and interaction, designing the control system for the player's actions. Additionally, Saad worked on adding sound effects and ensuring the game responded to keyboard inputs in real-time.

5. Project Implementation

Design and Structure:
The game was built around a simple grid in the console, where the *Flappy Bird* gameplay was rendered. We used the `gotoxy` function to update the grid in real-time, allowing dynamic positioning of the bird and other elements. The bird remained stationary in terms of vertical position, while the platforms (pipes) moved continuously to the left. Extended ASCII character 219 (█) was used as the block element to form the pipes and other obstacles. Sound effects and file handling were integrated later in the development process to enhance the game experience.

Functionalities Developed:

- Color customization for game elements

- Platform movement with random pipe position generation
- Dynamic bird movement with multiple bird variations for added variety

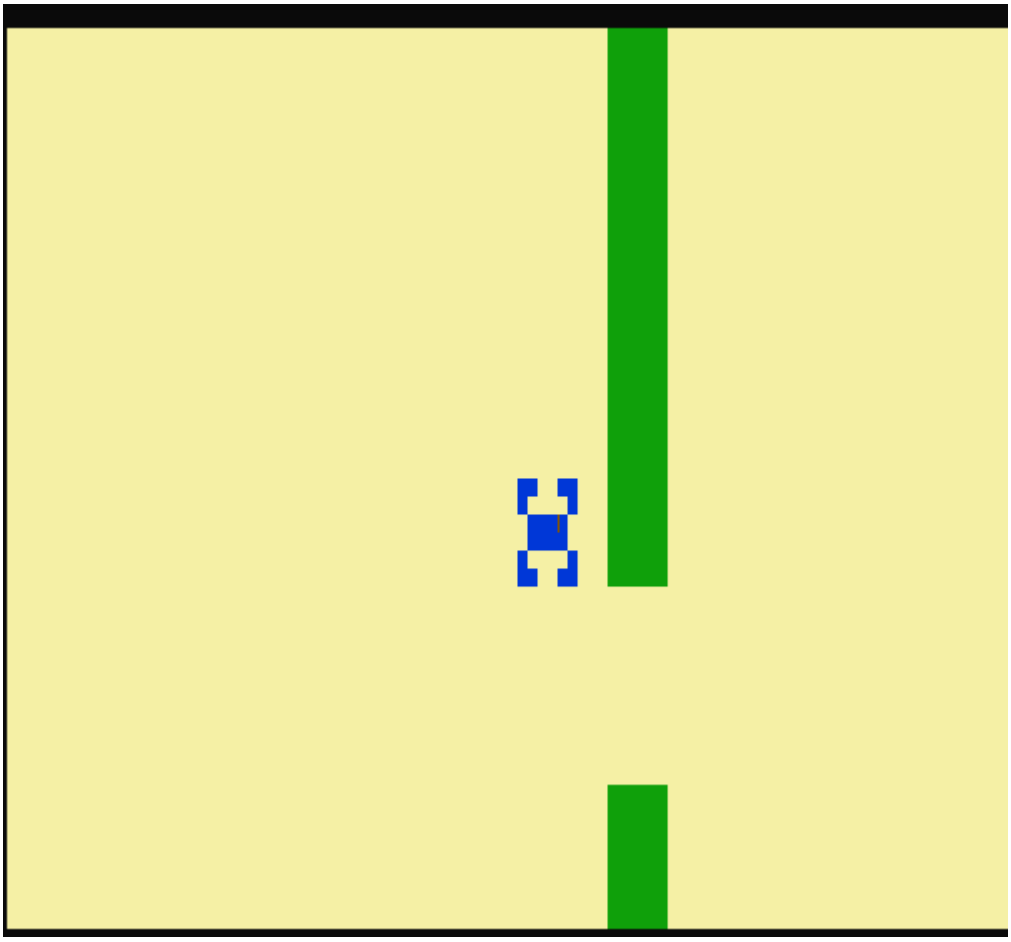
Challenges Faced:

- **Issue:** The screen flickered because we were using `WriteChar` for drawing, causing inefficient screen updates.
- **Solution:** Initially, we experimented with double buffering to resolve the flickering, but ultimately found that using `gotoxy` to position elements directly without clearing the screen was more effective in providing smooth gameplay.

6. Results

Project Outcomes:
The project will result in an engaging and entertaining *Flappy Bird* game, featuring unique elements such as character customization, score tracking, and in-game commentary. These features will enhance the gameplay experience significantly.

- **Screenshots and Illustrations**:



- **Testing and Validation**:We tested the game on multiple computers to ensure consistent performance and verify that the output remained the same across different systems. This helped identify any discrepancies in rendering or functionality.

7. Conclusion

Summary of Findings:

The project successfully recreated *Flappy Bird* in assembly language using the Irvine32 library. Key accomplishments included dynamic platform movement, random pipe generation, character customization, and score tracking. These features addressed the challenge of building a functional game in a low-level programming environment while staying within the limitations of a console-based interface.

Final Remarks:

This project provided valuable insights into low-level programming and game development. Challenges like screen flickering and platform movement were solved through experimentation with `gotoxy` and other techniques. Future improvements could focus on optimizing performance, enhancing sound, and refining the user interface. Overall, the project was a rewarding experience that demonstrated the power of assembly in creating interactive systems.